

```
File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'eggs'
```

Some commonly overwritten Python names are `all`, `any`, `date`, `email`, `file`, `format`, `hash`, `id`, `input`, `list`, `min`, `max`, `object`, `open`, `random`, `set`, `str`, `sum`, `test`, and `type`. Don't use these names for your identifiers.

Another common problem is naming your `.py` files the same names as third-party modules. For example, if you installed the third-party `Pyperclip` module but also created a `pyperclip.py` file, an `import pyperclip` statement imports `pyperclip.py` instead of the `Pyperclip` module. When you try to call `Pyperclip`'s `copy()` or `paste()` functions, you'll get an error saying they don't exist:

```
>>> # Run this code with a file named pyperclip.py in the current folder.
>>> import pyperclip # This imports your pyperclip.py, not the real one.
>>> pyperclip.copy('hello')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: module 'pyperclip' has no attribute 'copy'
```

Be aware of overwriting existing names in your Python code, especially if you're unexpectedly getting these `has no attribute` error messages.

The Worst Possible Variable Names Ever

The name `data` is a terrible, generic variable name, because literally all variables contain data. The same goes for naming variables `var`, which is a bit like naming your pet dog "Dog." The name `temp` is common for variables that temporarily hold data but is still a poor choice: after all, from a Zen perspective, all variables are temporary. Unfortunately, these names occur frequently despite their vagueness; avoid using them in your code.

If you need a variable to hold the statistical variance of your temperature data, please use the name `temperatureVariance`. It should go without saying that the name `tempVarData` would be a poor choice.

Summary